MULUNGUSHI UNIVERSITY (SCHOOL OF ENGINEERING AND TECHNOLOGY)

ICT202 – DATA STRUCTURES AND ALGORITHMS

Infix converter (infix to prefix / postfix)

STUDENT NAME: Nkonde. R. Zimba

STUDENT ID: 202402132

LECTURER: MR KALUBA

COURSE NAME: DATA STRUCTURES AND ALGORITHMS

COURSE CODE:202

YEAR OF STUDY: YEAR 2

GIVEN DATE: TUESDAY 15TH APRIL 2026

DUE DATE: THURSDAY 16th April 24.00hrs

Question

write a java program for converting an infinite expression into postfix and prefix: (a)workflow for your solution, (b)pseudo code, (c) java program

Workload Analysis

Converting an infix expression to postfix and prefix uses a stack-based algorithm. The analysis below outlines the time and space complexity of the solution.

Time Complexity

- Scan expression left to right: O(n)
- Each character is pushed/popped at most once: O (1) per character
- Overall Time Complexity: O(n), where n = number of characters

Space Complexity

- Stack stores at most n operators: O(n)
- Output string stores the result: O(n)
- Overall Space Complexity: O(n)

JUSTIFICATION

Every operand and operator is visited exactly once during the single left-to-right scan. Stack push and pop operations are O(1), so the total cost is proportional to the length of the expression — giving O(n) time overall. The stack and output buffer each require at most O(n) space.

(b) Pseudocode

INFIX TO POSTFIX

```
FUNCTION infixToPostfix(expression):
    CREATE empty stack
    CREATE empty output string

    FOR each character ch in expression:
        IF ch is an operand (letter/digit):
            APPEND ch to output
        ELSE IF ch is '(':
            PUSH ch onto stack
        ELSE IF ch is ')':
            WHILE stack top is not '(':
                POP from stack and APPEND to output
            POP '(' from stack (discard)
        ELSE IF ch is an operator (+, -, *, /):
            WHILE stack is not empty AND
                  precedence(stack top) >= precedence(ch):
                POP from stack and APPEND to output
            PUSH ch onto stack

    WHILE stack is not empty:
        POP from stack and APPEND to output

    RETURN output
END FUNCTION
```

INFIX TO PREFIX

```
FUNCTION infixToPrefix(expression):
    REVERSE the expression
    SWAP '(' with ')' and vice versa
    result = infixToPostfix(reversed expression)
    REVERSE result
    RETURN result
END FUNCTION
```

(C) **JAVA PROGRAM**

```
import java.util.Scanner;
import java.util.Stack;

public class InfixConverter {

  // Returns operator precedence
  static int precedence(char op) {
    if (op == '+' || op == '-') return 1;
    if (op == '*' || op == '/') return 2;
    if (op == '^') return 3;
    return 0;
  }

  // Infix to Postfix
  static String toPostfix(String expr) {
    Stack<Character> stack = new Stack<>();
    StringBuilder output = new StringBuilder();

    for (char ch : expr.toCharArray()) {
      if (Character.isLetterOrDigit(ch)) {
        output.append(ch);
      }
      else if (ch == '(') {
        stack.push(ch);
      }
      else if (ch == ')') {
        while (!stack.isEmpty() && stack.peek() != '(')
          output.append(stack.pop());
        if (!stack.isEmpty()) stack.pop();
      }
```

```
        else { // operator
            while (!stack.isEmpty() &&
                precedence(stack.peek()) >= precedence(ch))
                output.append(stack.pop());
            stack.push(ch);
        }
    }

    while (!stack.isEmpty())
        output.append(stack.pop());

    return output.toString();
}

// Infix to Prefix
static String toPrefix(String expr) {
    StringBuilder reversed = new StringBuilder();

    for (int i = expr.length() - 1; i >= 0; i--) {
        char ch = expr.charAt(i);
        if (ch == '(') reversed.append(')');
        else if (ch == ')') reversed.append('(');
        else reversed.append(ch);
    }

    String postfix = toPostfix(reversed.toString());
    return new StringBuilder(postfix).reverse().toString();
}

public static void main(String[] args) {
```

```
Scanner sc = new Scanner(System.in);

System.out.println("=== Infix Converter ===");
System.out.print("Enter infix expression: ");
String expr = sc.nextLine();

System.out.println("\nChoose conversion:");
System.out.println("1. Postfix");
System.out.println("2. Prefix");
System.out.println("3. Both");
System.out.print("Enter choice: ");

int choice = sc.nextInt();

System.out.println("\nInfix: " + expr);

switch (choice) {
  case 1:
    System.out.println("Postfix: " + toPostfix(expr));
    break;
  case 2:
    System.out.println("Prefix: " + toPrefix(expr));
    break;
  case 3:
    System.out.println("Postfix: " + toPostfix(expr));
    System.out.println("Prefix: " + toPrefix(expr));
    break;
  default:
    System.out.println("Invalid choice!");
}
```

```
        sc.close();
    }
}
```

SUMMARY /PRESENTATION OF OUTPUT RESULTS FROM JAVA PROGRAM

Infix Expression	Postfix Result	Prefix Result
A+B*C-D/E	ABC*+DE/-	-+A*BC/DE
(A+B)*(C-D)	AB+CD-*	*+AB-CD